

UNIVERSIDAD POLITÉCNICA SALESIANA DEL ECUADOR
SEDE QUITO – CAMPUS SUR
CARRERA DE COMPUTACIÓN
Programación y Plataformas Web

INFORME TÉCNICO

**Taller 3: Implementación de un Backend con Express.js y MongoDB,
Diseño de Endpoints REST y Modelado de Arquitectura mediante C4 —
Sistema GeoKipu**

Integrantes:

Juan David Figueroa
Christian Humberto Carrión
Jairo Alejandro Ojeda
Josué Samuel Vele

Docente:

Ing. Patsy Prieto

Quito, 13 de julio del 2026

Tabla de contenido

Tabla de contenido.....	2
1. Introducción.....	3
2. Objetivos.....	3
2.1 Objetivo general.....	3
2.2 Objetivos específicos.....	3
3. Marco teórico	3
3.1 Node.js y Express.js	3
3.2 MongoDB y Mongoose.....	3
3.3 Arquitectura REST.....	4
3.4 Cliente HTTP: Postman	4
3.5 Modelo C4.....	4
4. Desarrollo de la práctica	4
4.1 Conexión del backend a MongoDB	4
4.2 Endpoint de Autenticación — /api/auth.....	5
4.2.1 POST /auth/login	5
4.2.2 GET /auth/me.....	6
4.3 Endpoint 1: Usuarios — /api/users.....	7
4.3.1 POST /users.....	7
4.3.2 GET /users.....	8
4.4 Endpoint 2: Grupos — /api/groups	9
4.4.1 POST /groups.....	9
4.4.2 GET /groups.....	10
4.5 Funcionalidad complementaria: Integrantes de grupo — /api/groups/:groupId/members.....	11
4.6 Endpoint 3: Ubicaciones — /api/locations.....	13
4.6.1 POST /locations.....	13
4.6.2 GET /locations.....	14
4.7 Servicio adicional: estado del proveedor de mapas — /api/maps/status	15
4.8 Modelo C4 — Nivel de Contexto	16
5. Resultados frente a la rúbrica de evaluación.....	16
6. Conclusiones	17
Referencias	17

1. Introducción

El presente informe documenta el desarrollo del Taller 3 de la asignatura Programación y Plataformas Web, cuyo propósito es construir el backend de una aplicación utilizando el framework Express.js sobre el entorno de ejecución Node.js [1], [7], integrando una base de datos NoSQL mediante MongoDB [2] y el modelador de esquemas Mongoose [6]. El proyecto corresponde a la evolución del sistema GeoKipu (anteriormente denominado “Quito Ubicación Segura”), una aplicación académica orientada a compartir ubicación con consentimiento entre grupos familiares o de confianza.

En la práctica se migró la capa de persistencia desde un almacén en memoria hacia una base de datos MongoDB real, se implementaron endpoints REST siguiendo los principios de la arquitectura orientada a recursos descrita por Fielding [4], [8], y se verificó su correcto funcionamiento utilizando Postman como cliente HTTP [5]. Adicionalmente, se actualizó el modelo de arquitectura de software C4 [3] para reflejar el nuevo componente de persistencia.

2. Objetivos

2.1 Objetivo general

Diseñar e implementar un backend en Express.js con persistencia real en MongoDB, exponiendo endpoints REST verificados mediante un cliente HTTP, y representar su arquitectura utilizando el modelo C4.

2.2 Objetivos específicos

- Configurar la conexión entre el servidor Express y una base de datos MongoDB, evidenciando su estado mediante un endpoint de salud (health check).
- Implementar al menos tres endpoints, cada uno con dos métodos HTTP, respaldados por rutas, servicios/controladores y modelos NoSQL.
- Verificar el comportamiento de cada endpoint utilizando Postman, documentando solicitud y respuesta de cada prueba.
- Elaborar el modelo C4 de contexto del sistema GeoKipu, identificando actores, el sistema de software y los sistemas externos involucrados.

3. Marco teórico

3.1 Node.js y Express.js

Node.js es un entorno de ejecución de JavaScript del lado del servidor construido sobre el motor V8, orientado a operaciones de entrada/salidas no bloqueantes [7]. Express.js es un framework minimalista para Node.js que simplifica la creación de aplicaciones web y APIs mediante un sistema de enrutamiento (routes), middlewares y controladores [1].

3.2 MongoDB y Mongoose

MongoDB es un sistema de gestión de bases de datos NoSQL orientado a documentos, que almacena la información en colecciones de documentos BSON con esquemas flexibles [2]. Mongoose es una biblioteca de modelado de objetos (ODM) para Node.js que permite definir esquemas, validaciones y modelos que se traducen en colecciones dentro de MongoDB [6].

3.3 Arquitectura REST

La transferencia de estado representacional (REST) es un estilo arquitectónico para sistemas distribuidos basado en recursos identificados por URI y manipulados mediante métodos HTTP estándar, tal como se define en la especificación semántica de HTTP/1.1 [8] y en la disertación original de Fielding [4].

3.4 Cliente HTTP: Postman

Postman es una plataforma para el diseño, prueba y documentación de APIs que permite construir colecciones de solicitudes HTTP, definir variables de entorno y validar respuestas de manera reproducible [5].

3.5 Modelo C4

El modelo C4, propuesto por Simon Brown, es una técnica de diagramación de arquitectura de software basada en cuatro niveles jerárquicos —Contexto, Contenedores, Componentes y Código— que permite comunicar el diseño de un sistema a distintas audiencias [3].

4. Desarrollo de la práctica

4.1 Conexión del backend a MongoDB

El servidor Express se inicia mediante nodemon, el cual observa los archivos del proyecto y reinicia el proceso automáticamente ante cambios. Al arrancar, el backend establece conexión con la instancia local de MongoDB e informa por consola que la “Persistencia principal” se encuentra activa, además de indicar el puerto de escucha (4000).



```
PS C:\Users\aleja\Documents\ALEJANDRO OJEDA\UNIVERSIDAD\5to Semestre\app-ubicacion-quito\backend> npm run dev
> geokipu-backend@1.0.0 dev
> nodemon src/server.js

[nodemon] 3.1.14
[nodemon] to restart at any time, enter `rs`
[nodemon] watching path(s): *.*
[nodemon] watching extensions: js,mjs,cjs,json
[nodemon] starting `node src/server.js`
MongoDB conectado. Persistencia principal activa.
GeoKipu backend escuchando en http://localhost:4000
```

Fig. 1. Consola del servidor Express iniciando con nodemon y conexión activa a MongoDB.

Para verificar de forma independiente el estado del servicio, se ejecutó una petición GET hacia el endpoint `/api/health` utilizando el cmdlet `Invoke-RestMethod` de PowerShell. La respuesta confirma que el almacenamiento (storage) es mongodb y que el estado de la conexión (mongodb) es connected, junto con el tiempo de actividad del proceso (uptimeSeconds).



```
PS C:\Users\aleja\Documents\ALEJANDRO OJEDA\UNIVERSIDAD\5to Semestre\app-ubicacion-quito> Invoke-RestMethod http://localhost:4000/api/health
ok           : True
app          : GeoKipu
service      : backend
storage      : mongodb
mongodb      : connected
uptimeSeconds : 19
timestamp    : 2026-07-07T04:38:20.111Z
```

Fig. 2. Verificación de estado del backend mediante `Invoke-RestMethod` (health check).

Con el fin de organizar las pruebas, se configuró en Postman un entorno denominado “GeoKipu Local” con la variable `baseUrl` apuntando a `http://localhost:4000/api`, la cual es referenciada en todas las solicitudes de la colección mediante la sintaxis `{{baseUrl}}`.

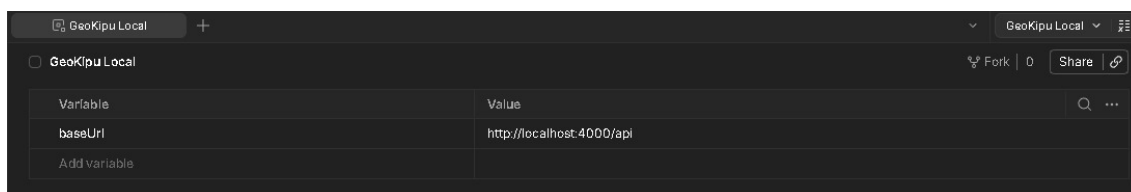


Fig. 3. Configuración de la variable de entorno `baseUrl` en el entorno Postman “GeoKipu Local”.

La petición `GET {{baseUrl}}/health`, ejecutada desde Postman, retorna un código de estado 200 OK con el mismo cuerpo JSON observado previamente, confirmando que la conexión a MongoDB permanece activa durante las pruebas.

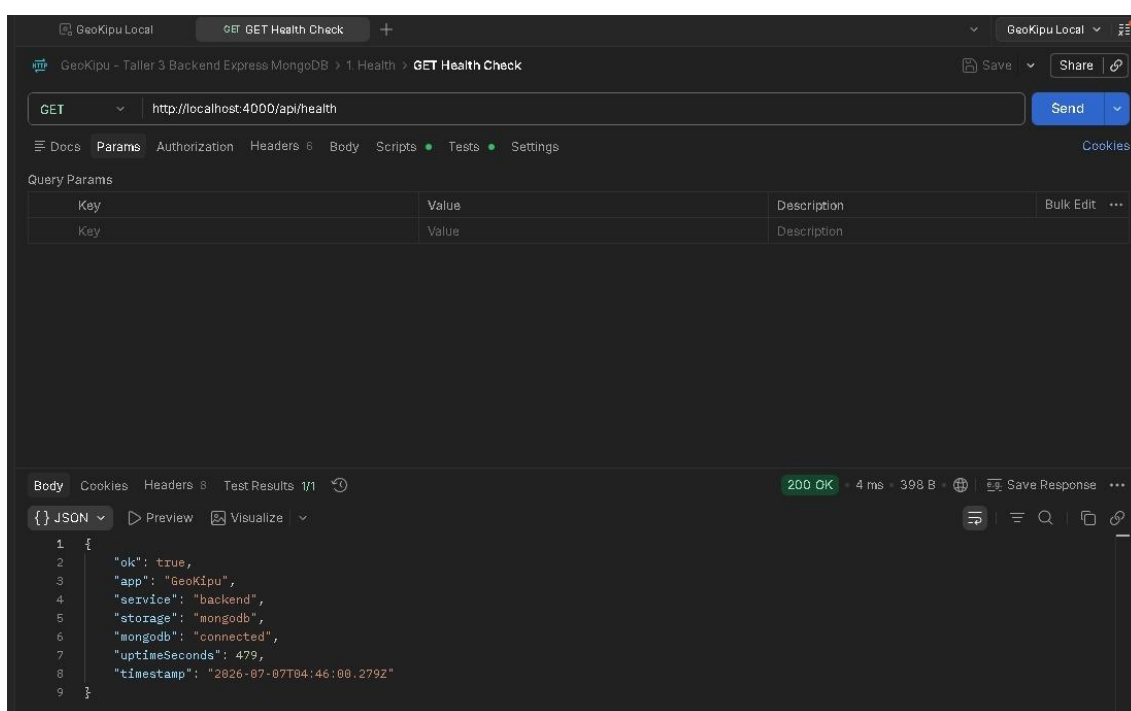


Fig. 4. Petición `GET /api/health` en Postman con respuesta 200 OK.

4.2 Endpoint de Autenticación — `/api/auth`

Este endpoint gestiona el inicio de sesión y la validación de sesión de los usuarios registrados. Implementa los métodos `POST /auth/login`, para autenticar a un usuario mediante correo y contraseña, y `GET /auth/me`, que retorna los datos del usuario autenticado a partir del token emitido.

4.2.1 `POST /auth/login`

Se autenticó al usuario administrador (`admin@geokipu.com`). El backend valida las credenciales contra la colección de usuarios en MongoDB y responde con un token simulado y el objeto de usuario, incluyendo campos persistidos como `createdAt` y `updatedAt` propios del modelo NoSQL.

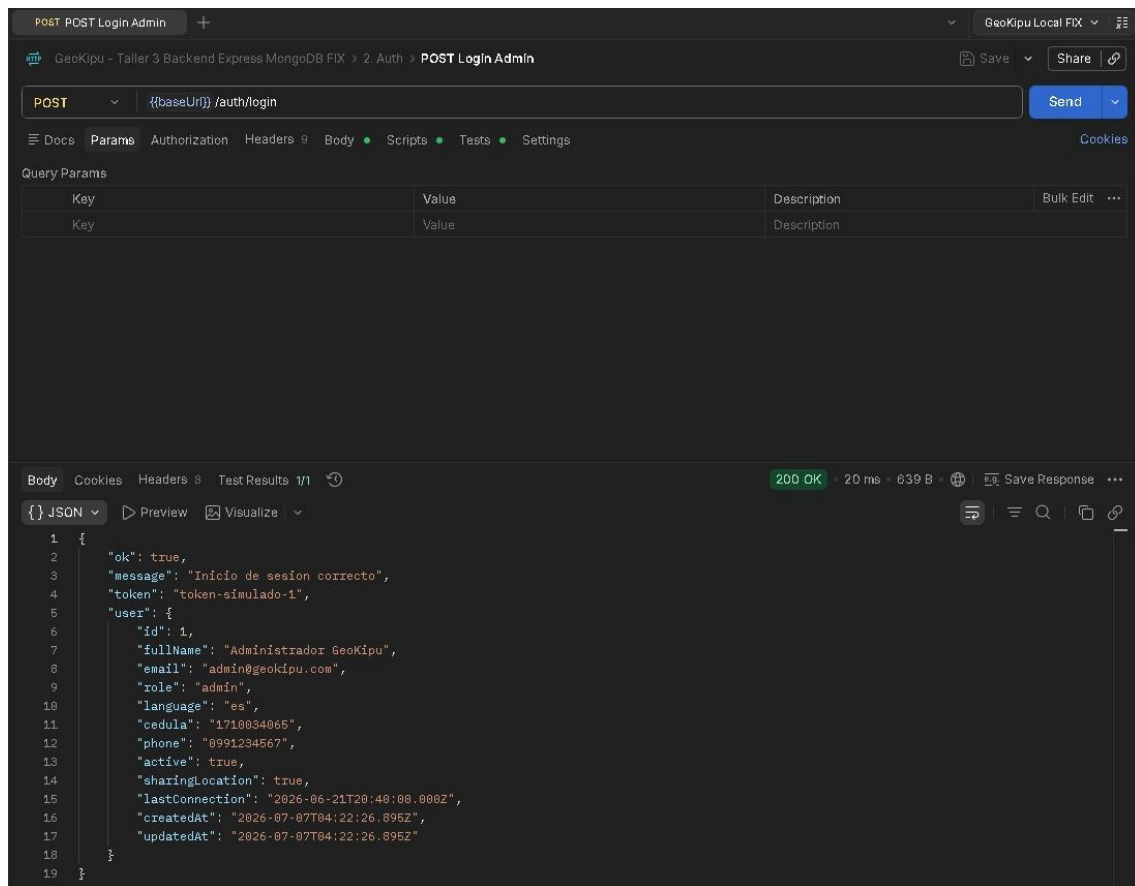


Fig. 5. Petición POST /auth/login para el usuario administrador.

4.2.2 GET /auth/me

Utilizando el token obtenido en el paso anterior, se consultó el endpoint /auth/me, el cual retorna el usuario autenticado junto con el indicador storage: mongodb, evidenciando que la información proviene de la base de datos y no de un arreglo en memoria.

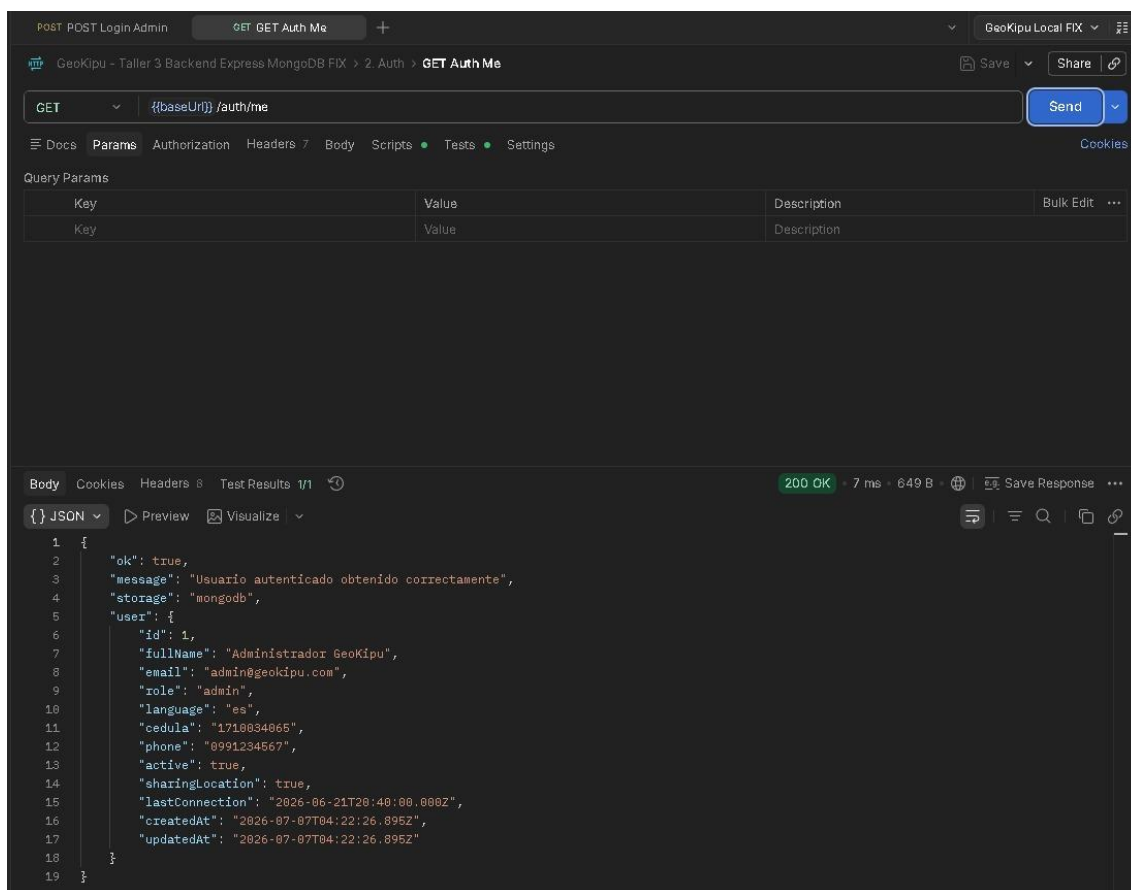


Fig. 6. Petición GET /auth/me con el token generado en el login.

4.3 Endpoint 1: Usuarios — /api/users

Este endpoint corresponde al primero de los tres endpoints principales solicitados por la práctica. Implementa los métodos POST /users, para el registro de nuevos usuarios, y GET /users, para listar los usuarios almacenados en la colección users de MongoDB.

4.3.1 POST /users

Se registró un nuevo usuario de rol persona (Alejandro Ojeda) enviando nombre, correo, teléfono, cédula, rol y contraseña. El backend responde con código 201 Created y el documento del usuario recién insertado, incluyendo el identificador asignado por la base de datos y las marcas de tiempo de creación.

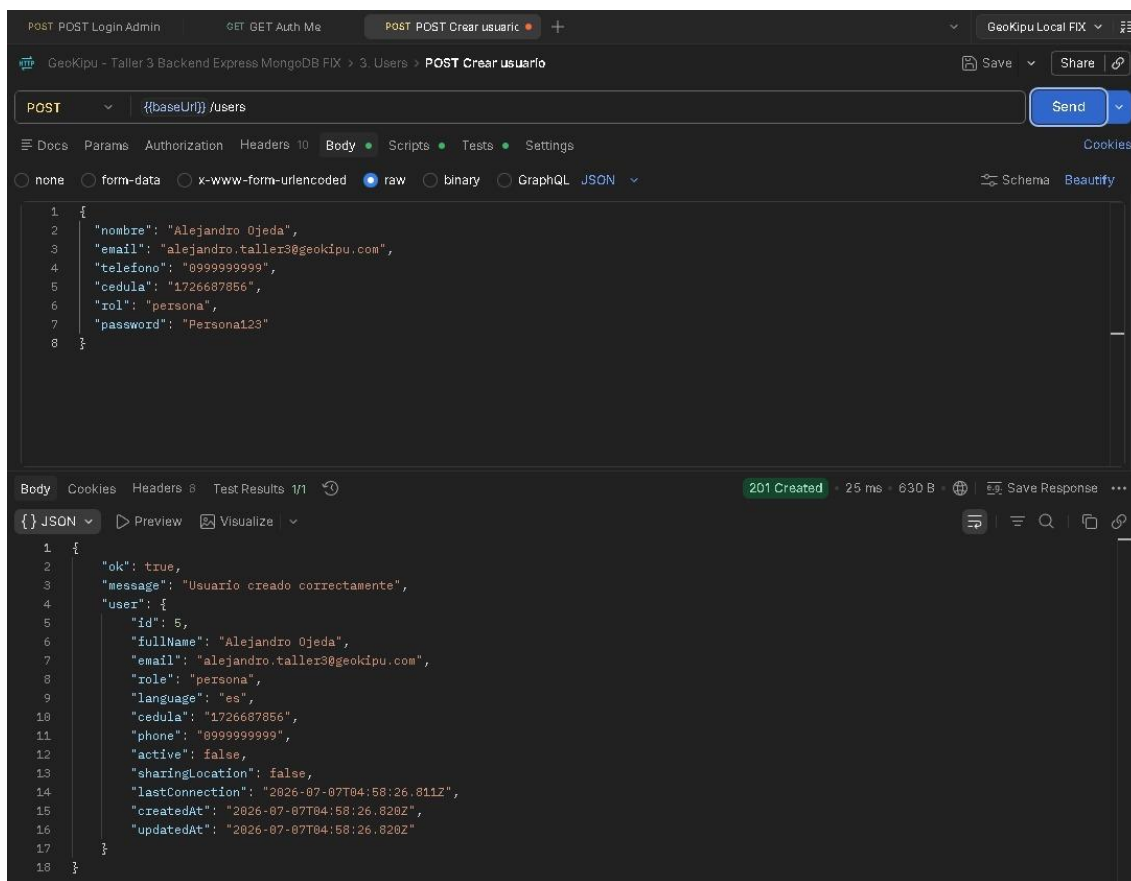


Fig. 7. Petición POST /users para el registro de un nuevo usuario tipo persona.

4.3.2 GET /users

La consulta GET /users retorna el listado completo de usuarios persistidos, incluyendo la última ubicación conocida (lastLocation) anidada para cada usuario, lo que demuestra la relación entre las colecciones users y locations.

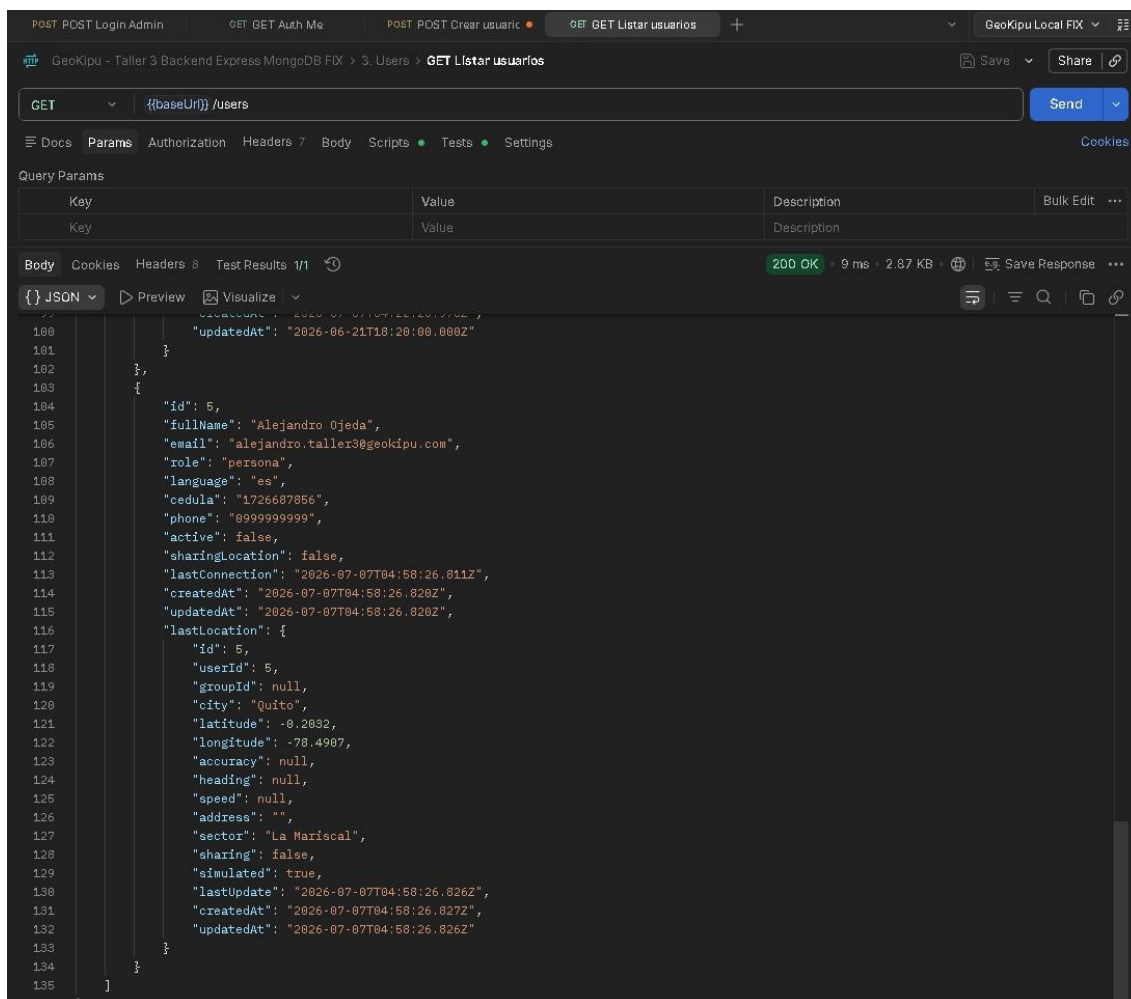


Fig. 8. Petición GET /users listando los usuarios registrados en MongoDB.

4.4 Endpoint 2: Grupos — /api/groups

El segundo endpoint principal gestiona los grupos de confianza. Implementa los métodos POST /groups, para crear un nuevo grupo, y GET /groups, para listar los grupos existentes junto con sus integrantes.

4.4.1 POST /groups

Se creó el grupo “Familia Ojeda”, quedando registrado con su creador (createdBy), fecha de creación y un arreglo members que incluye automáticamente al usuario creador.

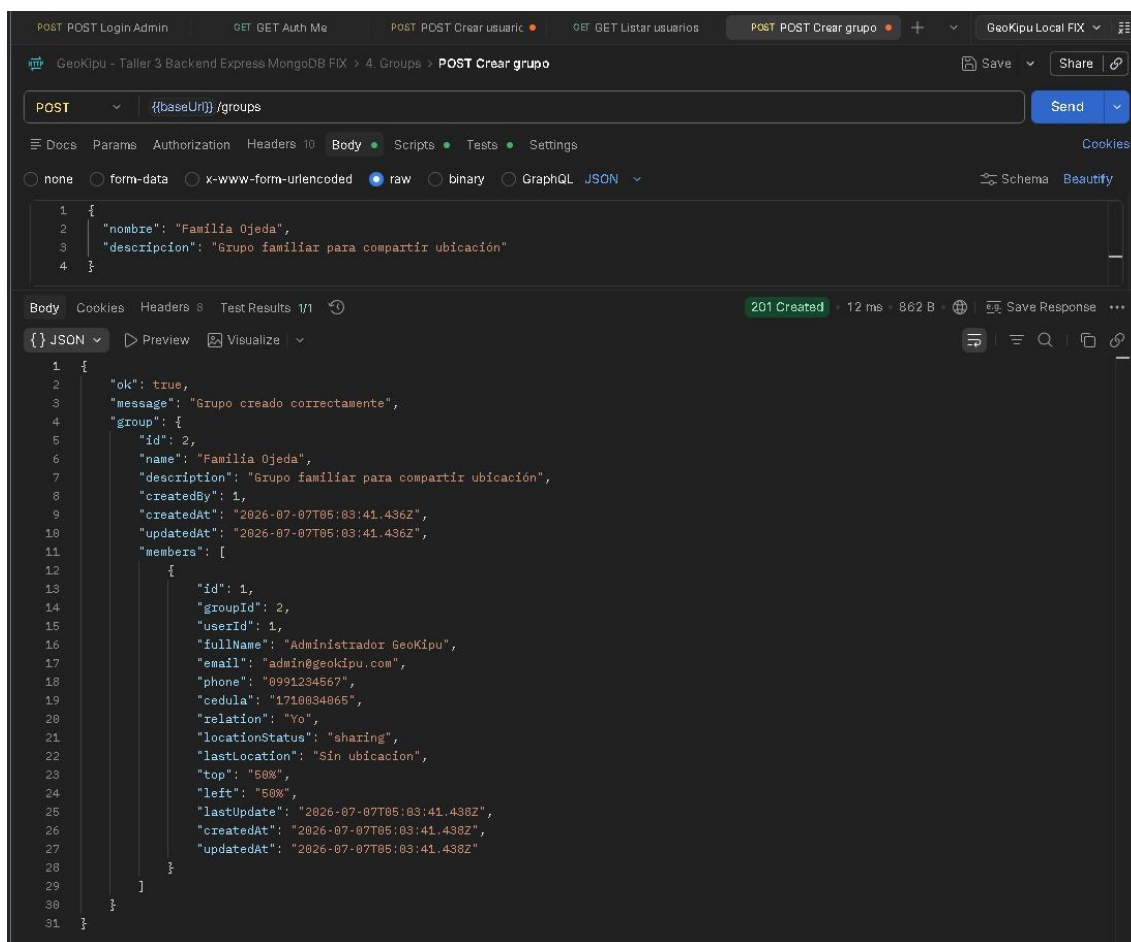


Fig. 9. Petición POST /groups para la creación de un grupo familiar.

4.4.2 GET /groups

La petición GET /groups devuelve el listado completo de grupos almacenados en MongoDB, cada uno con sus integrantes y el estado de ubicación (locationStatus) de cada miembro.

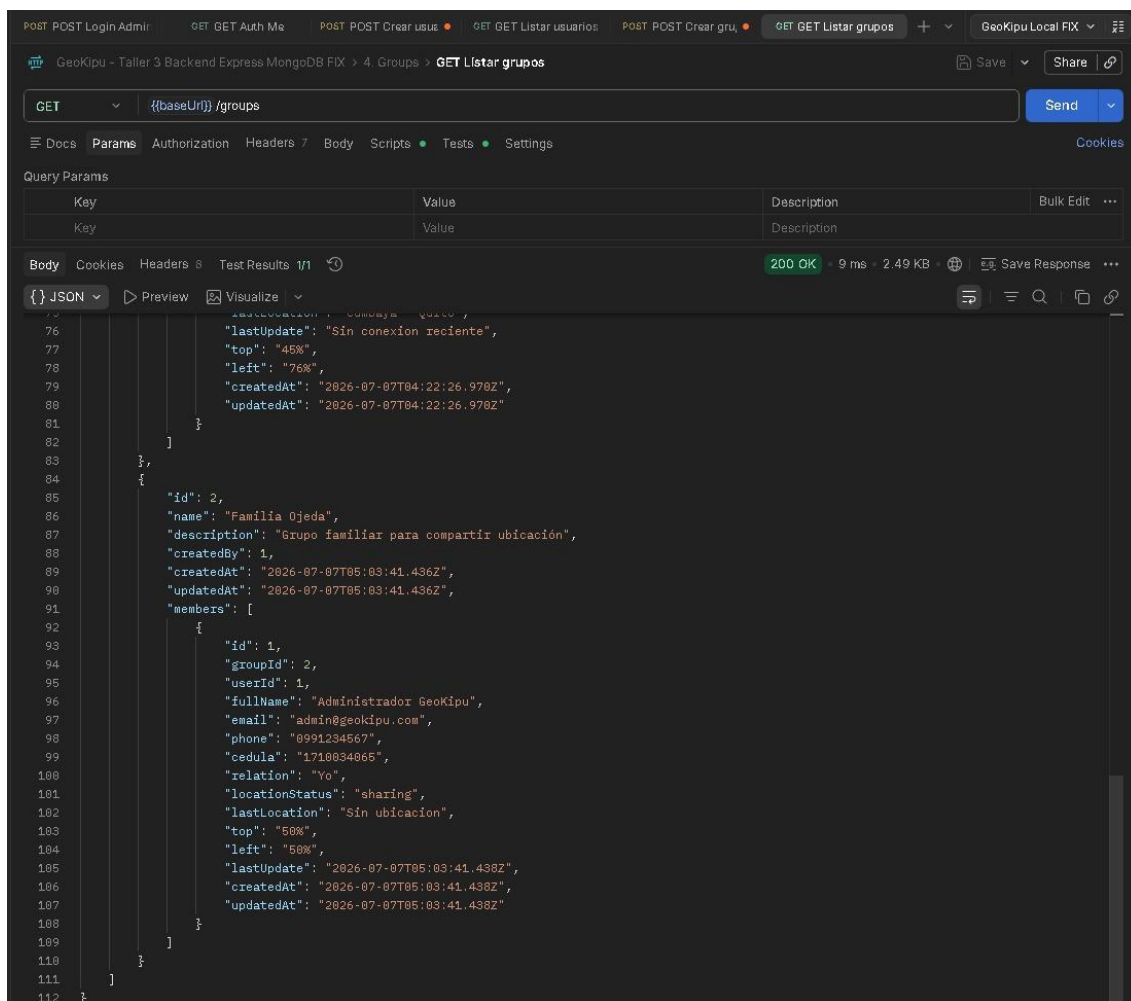


Fig. 10. Petición GET /groups listando los grupos existentes.

4.5 Funcionalidad complementaria: Integrantes de grupo — /api/groups/:groupId/members

Como extensión del endpoint de grupos, se implementó un subrecurso para administrar los integrantes de cada grupo, con los métodos POST /groups/:groupId/members, para agregar un miembro, y GET /groups/:groupId/members, para listarlos.

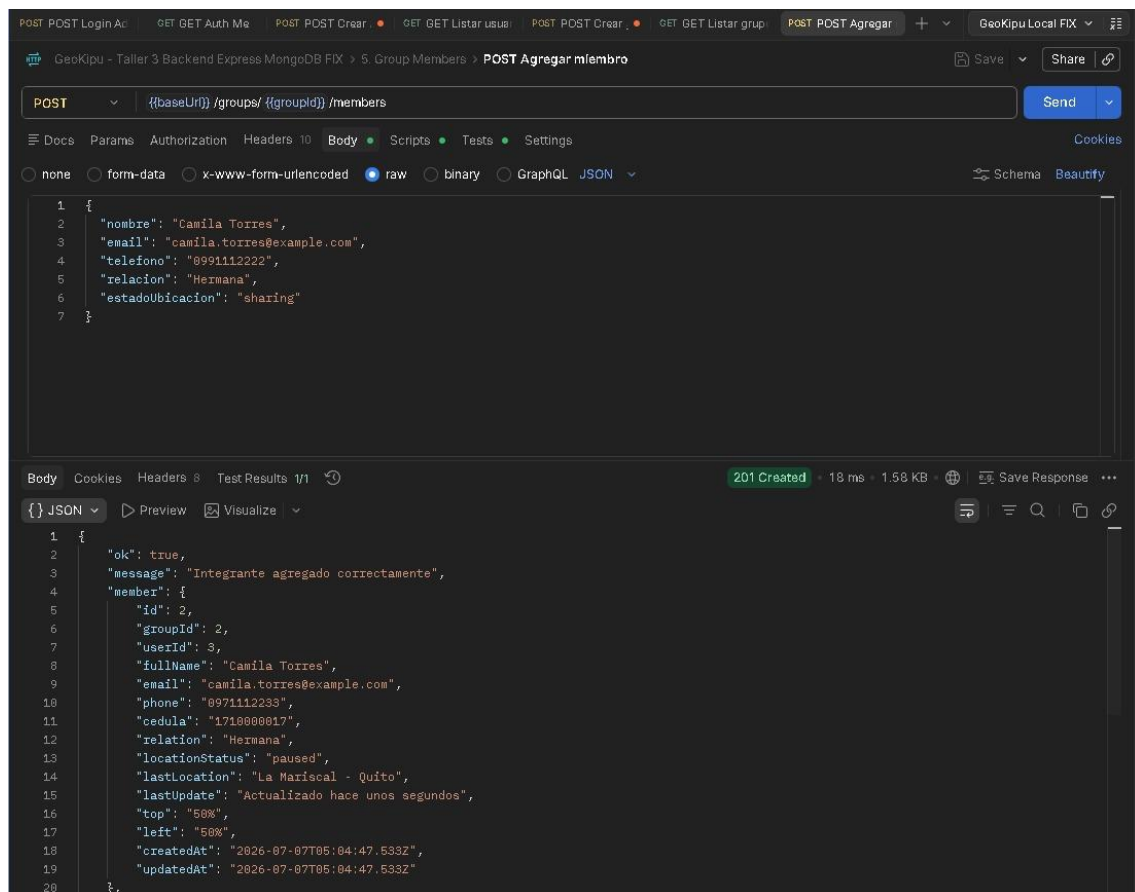


Fig. 11. Petición POST `/groups/:groupId/members` para agregar un integrante al grupo.

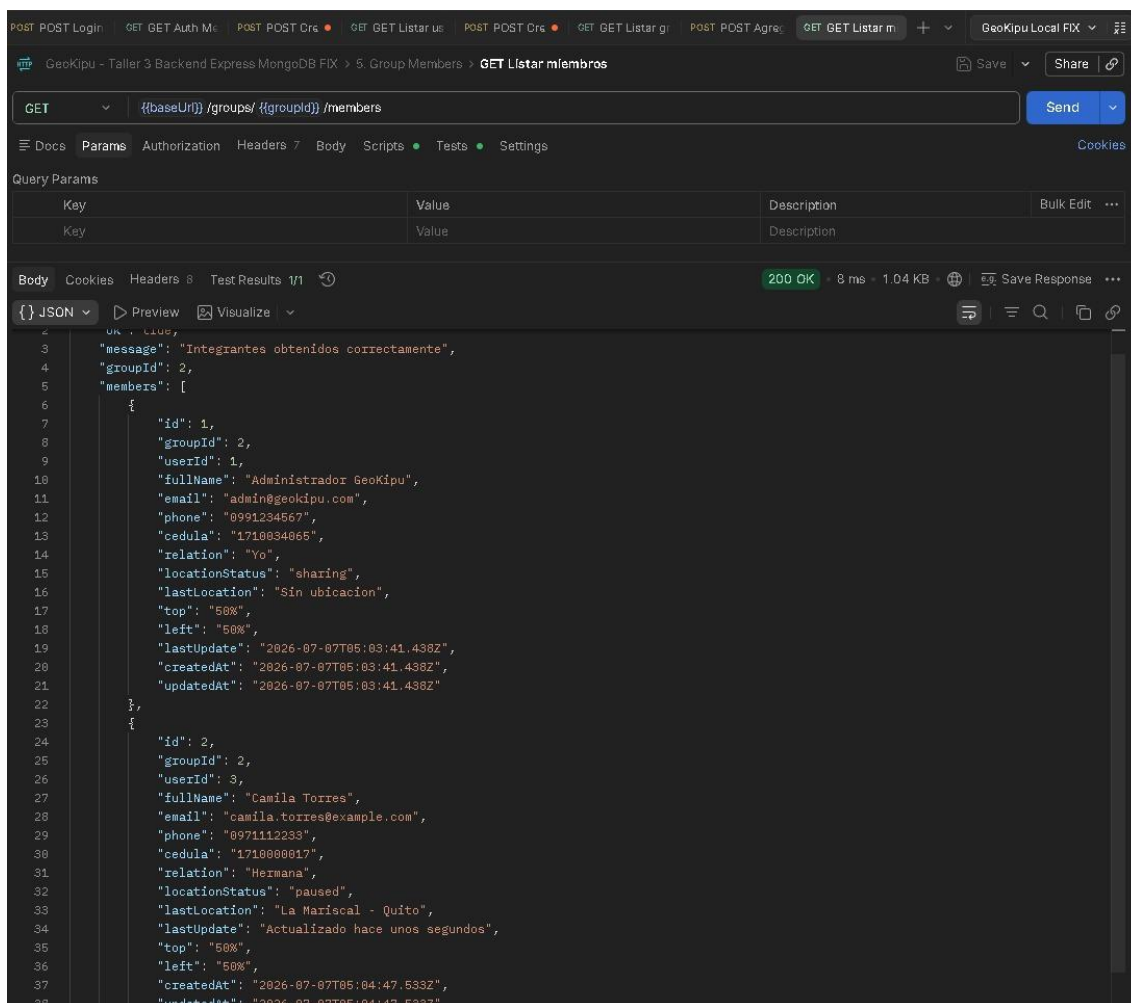


Fig. 12. Petición GET `/groups/{groupId}/members` listando los integrantes del grupo.

4.6 Endpoint 3: Ubicaciones — `/api/locations`

El tercer endpoint principal gestiona el registro y consulta de ubicaciones geográficas compartidas dentro de un grupo. Implementa los métodos POST `/locations`, para registrar una nueva ubicación, y GET `/locations`, para listar las ubicaciones almacenadas.

4.6.1 POST `/locations`

Se registró una ubicación en el sector La Carolina, Quito, asociada a un usuario y a un grupo específicos, con coordenadas, precisión (accuracy), dirección y estado de compartición (sharing). El backend responde con 201 Created y el documento persistido en la colección `locations`.

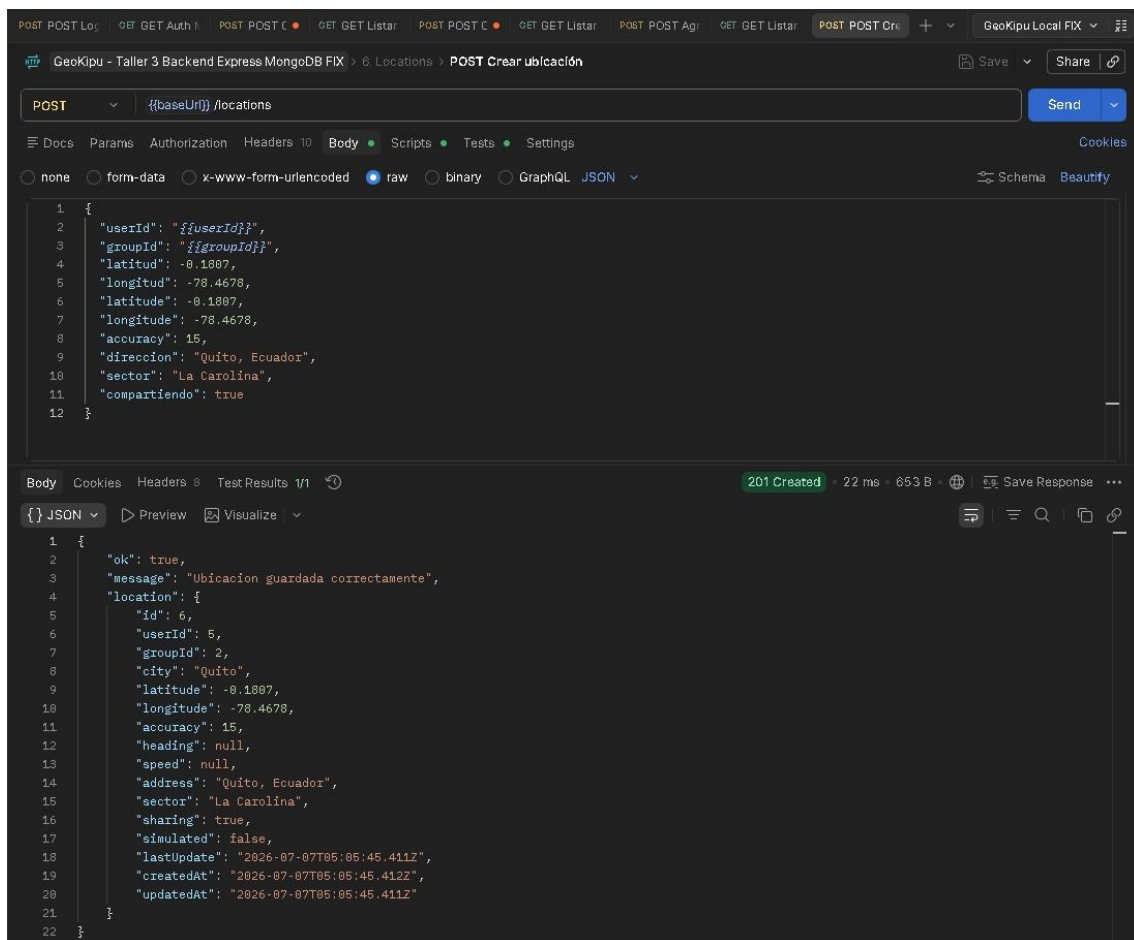


Fig. 13. Petición POST /locations para registrar una ubicación compartida.

4.6.2 GET /locations

La consulta GET /locations retorna el listado de ubicaciones almacenadas, junto con el indicador storage: mongodb y el conteo total de documentos (count), confirmando la persistencia de la información en la base de datos.

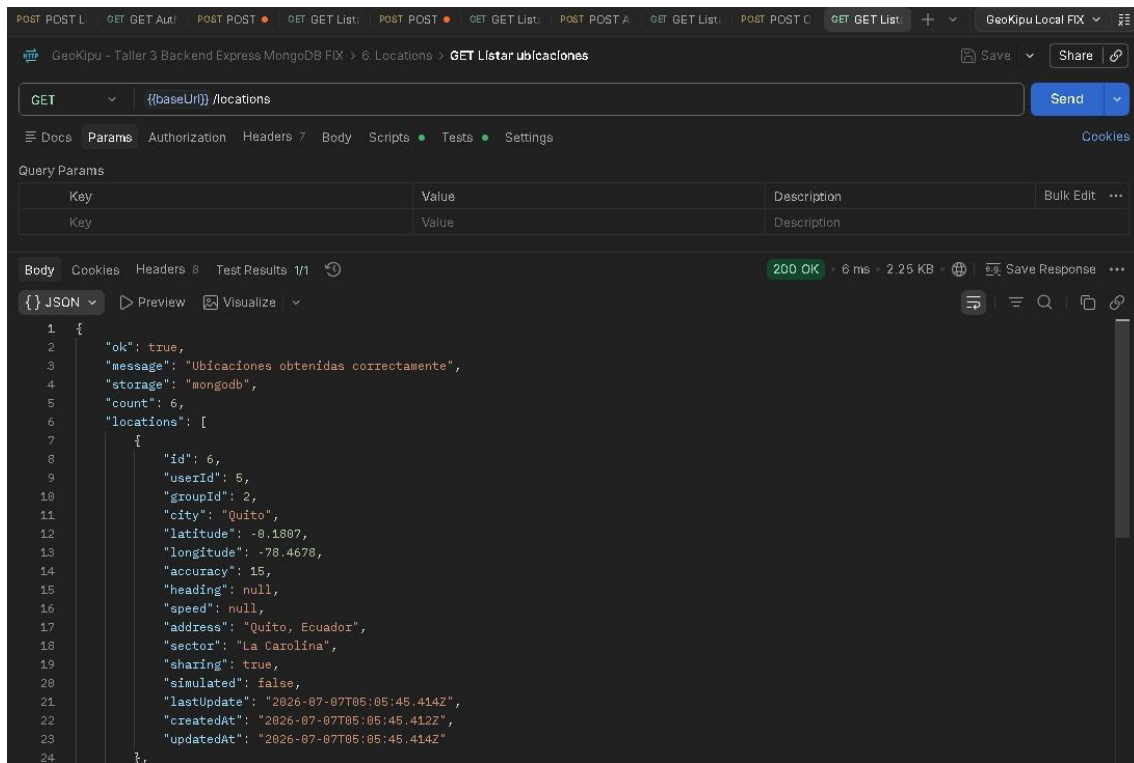


Fig. 14. Petición GET /locations listando las ubicaciones almacenadas.

4.7 Servicio adicional: estado del proveedor de mapas — /api/maps/status

Se implementó además un endpoint de diagnóstico para el proveedor externo de mapas (Geoapify), que informa si la clave de API se encuentra configurada y el modo de operación (real o simulado) del servicio de geocodificación.

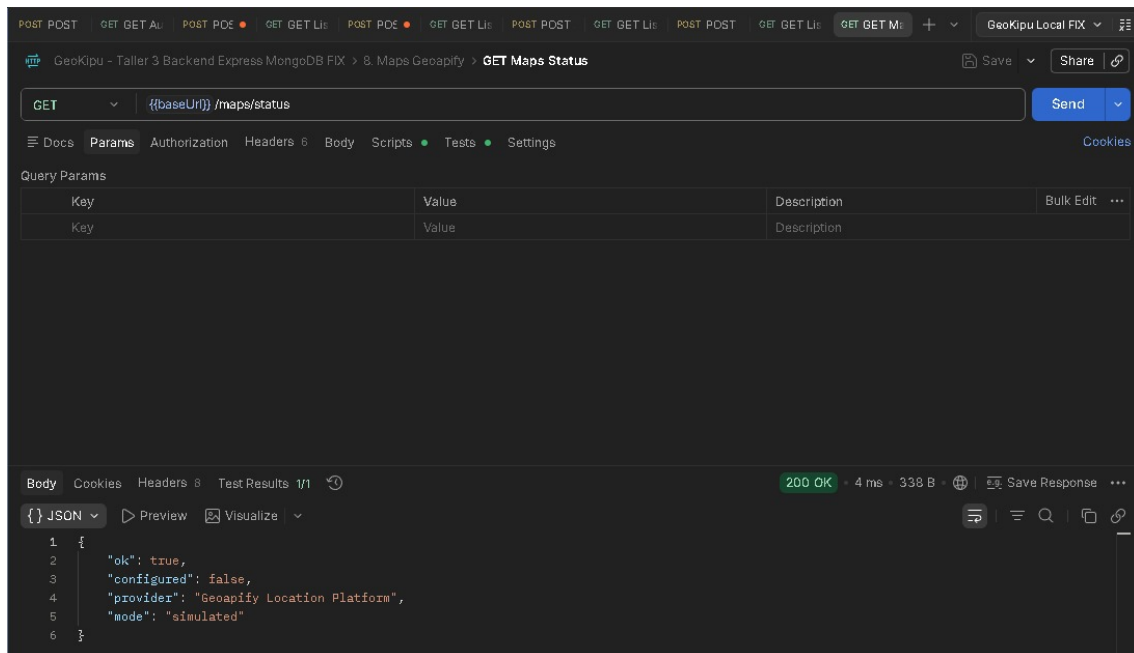


Fig. 15. Petición GET /maps/status para verificar el estado del proveedor Geoapify.

4.8 Modelo C4 — Nivel de Contexto

Con base en el desarrollo del backend, se actualizó el modelo C4 de contexto del sistema GeoKipu. El diagrama identifica dos actores (Administrador y Persona), el sistema de software GeoKipu y los sistemas externos con los que interactúa: Geoapify Location Platform, para servicios de mapas, y la propia base de datos MongoDB como mecanismo de persistencia.

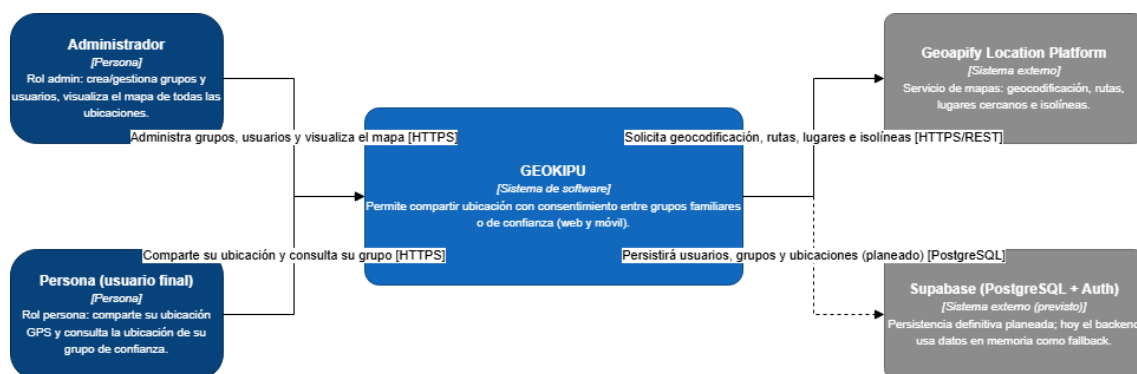


Fig. 16. Modelo C4 de contexto del sistema GeoKipu.

El Administrador gestiona grupos y usuarios y visualiza el mapa de todas las ubicaciones, mientras que la Persona comparte su ubicación GPS y consulta la de su grupo de confianza. El sistema GeoKipu consume el servicio externo Geoapify para geocodificación, rutas, lugares e isolíneas, y persiste su información en MongoDB, tal como se evidenció en los registros de conexión y en las respuestas JSON de cada endpoint.

5. Resultados frente a la rúbrica de evaluación

A continuación, se relaciona cada criterio de la rúbrica de calificación con la evidencia presentada en este informe.

Criterio de la rúbrica	Evidencia dentro del informe	Nivel alcanzado
Funcionalidad del endpoint en el backend	Existen routes (auth, users, groups, group-members, locations, maps), servicios/controladores y modelos Mongoose para cada colección. El backend responde correctamente en Fig. 4 a Fig. 15, con persistencia real en MongoDB (Fig. 1 y Fig. 2).	3 / 3 — más del 81 % funcional
Implementación de servicios	Se implementaron y probaron servicios para los tres endpoints solicitados (Usuarios, Grupos y Ubicaciones), más los endpoints de apoyo de Autenticación, Miembros de grupo y Maps/Geoapify (Fig. 5 a Fig. 15).	3 / 3 — tres endpoints con servicio
Conexión hacia la base de datos	El log del servidor confirma “MongoDB conectado. Persistencia principal activa” (Fig. 1) y el endpoint /api/health reporta storage: mongodb y mongodb: connected (Fig. 2 y Fig. 4). Los documentos devueltos (Fig. 8, 10, 12 y 14) corresponden a un modelo NoSQL coherente con colecciones users, groups y locations.	3 / 3 — conexión > 81 %, modelo NoSQL correcto

6. Conclusiones

- Se logró migrar la capa de persistencia del backend de un almacenamiento en memoria hacia MongoDB, evidenciando la conexión activa tanto en los registros de consola como en las respuestas del endpoint de salud.
- Se implementaron tres endpoints principales (Usuarios, Grupos y Ubicaciones), cada uno con al menos dos métodos HTTP, además de los endpoints de apoyo de Autenticación, Integrantes de grupo y Estado de mapas, todos respaldados por rutas, servicios y modelos NoSQL.
- El uso de Postman como cliente HTTP permitió documentar de manera reproducible el comportamiento de cada endpoint, incluyendo cuerpos de solicitud, códigos de estado y cuerpos de respuesta.
- El modelo C4 de contexto facilitó comunicar de forma clara los límites del sistema GeoKipu, sus actores y sus dependencias externas, sirviendo como base para futuros niveles de detalle (contenedores y componentes).
- El trabajo colaborativo entre los integrantes del equipo permitió distribuir la implementación de los distintos endpoints y consolidar un backend funcional dentro del tiempo previsto para la práctica.

Referencias

- [1] OpenJS Foundation, “Express - Node.js web application framework,” Express.js Documentation, 2024. [En línea]. Disponible: <https://expressjs.com/>
- [2] MongoDB, Inc., “MongoDB Manual,” MongoDB Docs, 2024. [En línea]. Disponible: <https://www.mongodb.com/docs/>
- [3] S. Brown, “The C4 model for visualising software architecture,” C4model.com, 2018. [En línea]. Disponible: <https://c4model.com/>
- [4] R. T. Fielding, “Architectural Styles and the Design of Network-based Software Architectures,” Tesis doctoral, Univ. of California, Irvine, CA, EE. UU., 2000.
- [5] Postman, Inc., “Postman API Platform Documentation,” Postman Learning Center, 2024. [En línea]. Disponible: <https://learning.postman.com/>
- [6] Automattic, Inc., “Mongoose ODM v8.x Documentation,” 2024. [En línea]. Disponible: <https://mongoosejs.com/docs/>
- [7] OpenJS Foundation, “Node.js Documentation,” 2024. [En línea]. Disponible: <https://nodejs.org/en/docs>
- [8] R. Fielding, J. Reschke, Eds., “Hypertext Transfer Protocol (HTTP/1.1): Semantics and Content,” RFC 7231, Internet Engineering Task Force (IETF), jun. 2014.